МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФГБОУ ВО «Донской государственный технический университет»

Факультет «Информатика и вычислительная техника»

Кафедра «Кибербезопасность информационных систем»

ОТЧЁТ

по лабораторной работе № 2

по дисциплине «Разработка защищённых Web-ресурсов»

«Разработка Web-приложения для расчёта теоретической температуры горения»

Разработка страницы расчёта температуры

Выполнил: обучающийся гр. ВКБ53

Ковалев Данил Петрович

Проверил: Ковальчик Р. В.

Ростов-на-Дону, 2026

1. Цель работы и постановка задачи

Цель работы — дополнить веб-приложение, созданное в лабораторной работе № 1, второй страницей с формой ввода параметров дутья, выводом результата расчёта теоретической температуры горения и сохранением выполненных расчётов в файл.

По заданию необходимо вынести формулу в отдельный Python-модуль, реализовать шаблон tempr.html и обработку GET/POST на маршруте /tempr, добавить необязательный комментарий и журнал static/calc_log.txt. Правильность формулы проверяется по четырём контрольным наборам из методических указаний. В завершение требуется ответить на пять контрольных вопросов.

В качестве основы использован проект лабораторной № 1 из соседнего каталога 2026/1. Его исходники и конфигурации перенесены в каталог 2026/2. Сохранены справочная таблица главной страницы, семантическая разметка, оформление, src-структура и внедрение зависимостей Dishka.

Работа выполнена локально с Python 3.14.3 и Flask 3.1.3. Зависимости установлены через uv в отдельное окружение .venv. Проверка страниц выполнена в Google Chrome через Playwright по адресу http://127.0.0.1:5002. Docker не собирался и не запускался.

```
uv sync --frozen --group report
uv run blast-furnace run --host 127.0.0.1 --port 5002
```

Листинг 1 — подготовка окружения и локальный запуск

2. Предметная область и расчётная формула

Теоретическая температура горения — расчётная температура продуктов сгорания кокса в фурменном очаге доменной печи при условии, что выделенное тепло полностью расходуется на их нагрев. В модели не учитываются теплопотери и теплообмен с шихтой. Для оперативной оценки применяется эмпирическая зависимость из методических указаний.

$$Tг = 2405 + 0{,}75 \cdot (Tд - 1200) - 5{,}40 \cdot Wд + 43{,}00 \cdot (w - 21) - 52{,}00 \cdot Vпг.$$

Параметр	Обозначение	Единица	Диапазон
Температура дутья	Тд	°C	900–1300
Влажность дутья	Wд	г/м³	1–40
Содержание кислорода	w	%	21–35
Расход природного газа	Vпг	м³/100 м³ дутья	0–15

Горячее дутьё повышает температуру, поскольку вносит дополнительное тепло. Влага и природный газ понижают её из-за эндотермических процессов разложения. Обогащение кислородом повышает температуру: уменьшается доля балластного азота, на нагрев которого расходуется тепло.

№	Тд	Wд	w	Vпг	Тг, °C
1	1000	10	21	0	2201,0
2	1200	15	25	8	2080,0
3	1100	20	23	5	2048,0
4	1300	1	35	0	3076,6

Например, для первого набора: $Tг = 2405 + 0{,}75 \cdot (1000 - 1200) - 5{,}4 \cdot 10 + 43 \cdot (21 - 21) - 52 \cdot 0 = 2405 - 150 - 54 = 2201{,}0$ °C. Все четыре результата совпали с контрольными значениями.

3. Структура проекта и бизнес-логика

Формула размещена в domain/services/calculations.py. Пакет services с пустым __init__.py соответствует требованию методички; он находится внутри domain для сохранения архитектуры первой лабораторной. Функция не зависит от Flask, HTML и файловой системы, поэтому проверяется отдельно от веб-сервера.

```
src/blast_furnace/
  app.py                          WSGI-точка входа
  domain/
    reference.py                  справочник из ЛР1
    temperature.py                параметры и диапазоны
    services/
      __init__.py                 пустой файл пакета
      calculations.py             формула calculate_ttg
  application/
    get_blast_reference.py         справочник
    calculate_temperature.py      расчёт и порт журнала
  infrastructure/
    calculation_log.py            запись UTF-8 файла
  presentation/http/
    routes.py                     обработчики HTTP
    templates/                    base, index, tempr
    static/styles.css             оформление
  setup/                          конфигурация и Dishka
static/calc_log.txt               журнал, создаётся при расчёте
tests/                            модульные и HTTP-тесты
docs/                             отчёт и подтверждения
```

Листинг 2 — структура дополненного проекта

```
def calculate_ttg(t_blast: float, humidity: float, oxygen: float, gas: float) -> float:
    """Теоретическая температура горения, °C (формула ЛР2)."""
    return (
        2405
        + 0.75 * (t_blast - 1200)
        - 5.40 * humidity
        + 43.00 * (oxygen - 21)
        - 52.00 * gas
    )
```

Листинг 3 — независимая функция расчёта

Модель BlastParameters проверяет допустимые диапазоны и конечность чисел. Прикладной сценарий CalculateTemperature вызывает формулу и порт CalculationLog. Конкретный файловый адаптер подключается в setup через Dishka; внутренние слои не импортируют Flask или инфраструктуру.

4. Обработка формы и передача результата

GET /tempr возвращает пустую форму. POST читает поля t_blast, humidity, oxygen, gas и comment из request.form. Числа преобразуются из строк в float, что позволяет вводить дробные значения. Запятая заменяется точкой. Ошибки преобразования и выхода за диапазон дают HTTP 400; введённые значения сохраняются в форме. Комментарий необязателен, его длина ограничена 500 символами.

```
from decimal import Decimal

from dishka.integrations.flask import FromDishka, inject
from flask import Blueprint, current_app, render_template, request

from blast_furnace.application.calculate_temperature import CalculateTemperature
from blast_furnace.application.get_blast_reference import GetBlastReference
from blast_furnace.domain.temperature import BlastParameters

pages = Blueprint("pages", __name__, template_folder="templates", static_folder="static")


@pages.app_template_filter("decimal_ru")
def decimal_ru(value: Decimal) -> str:
    return str(value).replace(".", ",")


@pages.get("/")
@inject
def index(query: FromDishka[GetBlastReference]) -> str:
    return render_template("index.html", effects=query.execute())


@pages.route("/tempr", methods=["GET", "POST"])
@inject
def temperature(command: FromDishka[CalculateTemperature]) -> tuple[str, int]:
    values = {
        name: request.form.get(name, "")
        for name in ("t_blast", "humidity", "oxygen", "gas", "comment")
    }
    result = None
    error = None
    status = 200
    if request.method == "POST":
        try:
            numbers = []
            for name, label in (
                ("t_blast", "Температура дутья"),
                ("humidity", "Влажность дутья"),
                ("oxygen", "Содержание кислорода"),
                ("gas", "Расход природного газа"),
            ):
                try:
                    numbers.append(float(values[name].strip().replace(",", ".")))
                except ValueError:
                    raise ValueError(f"{label}: введите число.") from None
            parameters = BlastParameters(*numbers)
            if len(values["comment"]) > 500:
                raise ValueError("Комментарий должен содержать не более 500 символов.")
            result = command.execute(parameters, values["comment"])
        except ValueError as exc:
            error, status = str(exc), 400
        except OSError:
            current_app.logger.exception("Не удалось сохранить расчёт")
            error = "Не удалось сохранить расчёт в журнал. Повторите попытку позже."
            status = 503
    return render_template("tempr.html", values=values, tempr=result, error=error), status
```

Листинг 4 — контроллер страниц и расчёта

5. HTML-форма и журнал расчётов

Шаблон tempr.html наследует base.html с блоками header, nav, main и footer. Каждое поле связано с label через id/for и имеет name для передачи в контроллер. Обязательные параметры имеют required и подсказки диапазонов. type=text и inputmode=decimal позволяют вводить десятичную запятую. Серверная проверка выполняется независимо от проверки браузера.

```
<form action="{{ url_for('pages.temperature') }}" method="POST">
  <label for="t_blast">Температура дутья, °C</label>
  <input type="text" inputmode="decimal" id="t_blast"
         name="t_blast" value="{{ values.t_blast }}" required>
  <!-- Аналогично: humidity, oxygen, gas -->
  <label for="comment">Комментарий к расчёту</label>
  <input type="text" id="comment" name="comment"
         value="{{ values.comment }}" maxlength="500">
  <button type="submit">Рассчитать</button>
</form>
{% if tempr is not none %}
  <output>{{ '%.1f' | format(tempr) | replace('.', ',') }}</output> °C
  <p>Комментарий: {{ values.comment or 'Не указан' }}</p>
{% endif %}
```

Листинг 5 — сокращённый фрагмент формы и вывода результата

Результат передаётся как tempr=result в render_template. Условие tempr is not none скрывает блок при первом открытии и корректно отличает отсутствие результата от числового нуля. Jinja автоматически экранирует комментарий и значения полей: введённые HTML-теги отображаются как текст.

В соответствии с заданием журнал ведётся в static/calc_log.txt. Режим a сохраняет предыдущие записи. Каждая строка содержит время с часовым поясом, четыре параметра, результат и комментарий. Каталог создаётся автоматически. Путь настраивается CALC_LOG_PATH. Файл журнала находится вне каталога статических ресурсов Blueprint и не доступен по HTTP.

5.1. Реализация записи в файл

```
import fcntl
from datetime import datetime
from pathlib import Path

from blast_furnace.domain.temperature import BlastParameters


class FileCalculationLog:
    def __init__(self, path: Path) -> None:
        self._path = path

    def append(self, parameters: BlastParameters, result: float, comment: str) -> None:
        # Экранируем разделители: пользователь не может подделать новую запись.
        comment = (
            comment.replace("\\", "\\\\")
            .replace("\r", "\\r")
            .replace("\n", "\\n")
            .replace("|", "\\|")
        )
        line = (
            f"{datetime.now().astimezone().isoformat(timespec='seconds')} | "
            f"Тд={parameters.t_blast:g} Wд={parameters.humidity:g} "
            f"w={parameters.oxygen:g} Vпг={parameters.gas:g} | "
            f"Tг={result:.1f} | {comment}\n"
        )
        self._path.parent.mkdir(parents=True, exist_ok=True)
        with self._path.open("a", encoding="utf-8") as stream:
            fcntl.flock(stream.fileno(), fcntl.LOCK_EX)
            stream.write(line)
            stream.flush()
            # Закрытие файла освобождает блокировку, в том числе при исключении.
```

Листинг 6 — файловый журнал расчётов

Блокировка fcntl предотвращает перемешивание строк при конкурентных записях в Linux/macOS. Переносы строк и разделители комментария экранируются для сохранения формата «одна запись — одна строка». Если файл недоступен, приложение возвращает HTTP 503 и сообщение об ошибке сохранения, не показывая успешный результат.

```
2026-11-13T16:32:44+03:00 | Тд=1000 Wд=10 w=21 Vпг=0 | Tг=2201.0 | Контрольный расчёт № 1 из
методички
2026-11-13T16:32:44+03:00 | Тд=1200 Wд=15 w=25 Vпг=8 | Tг=2080.0 | Контрольный расчёт № 2 из
методички
2026-11-13T16:32:45+03:00 | Тд=1100 Wд=20 w=23 Vпг=5 | Tг=2048.0 | Контрольный расчёт № 3 из
методички
2026-11-13T16:32:45+03:00 | Тд=1300 Wд=1 w=35 Vпг=0 | Tг=3076.6 | Контрольный расчёт № 4 из
методички
```

Листинг 7 — реальные записи четырёх контрольных расчётов

6. Проверка формы в браузере

Из главной страницы выполнен переход по ссылке «Расчёт ТТГ». При первом открытии отображаются четыре обязательных числовых поля, необязательный комментарий и кнопка расчёта; блока результата нет.

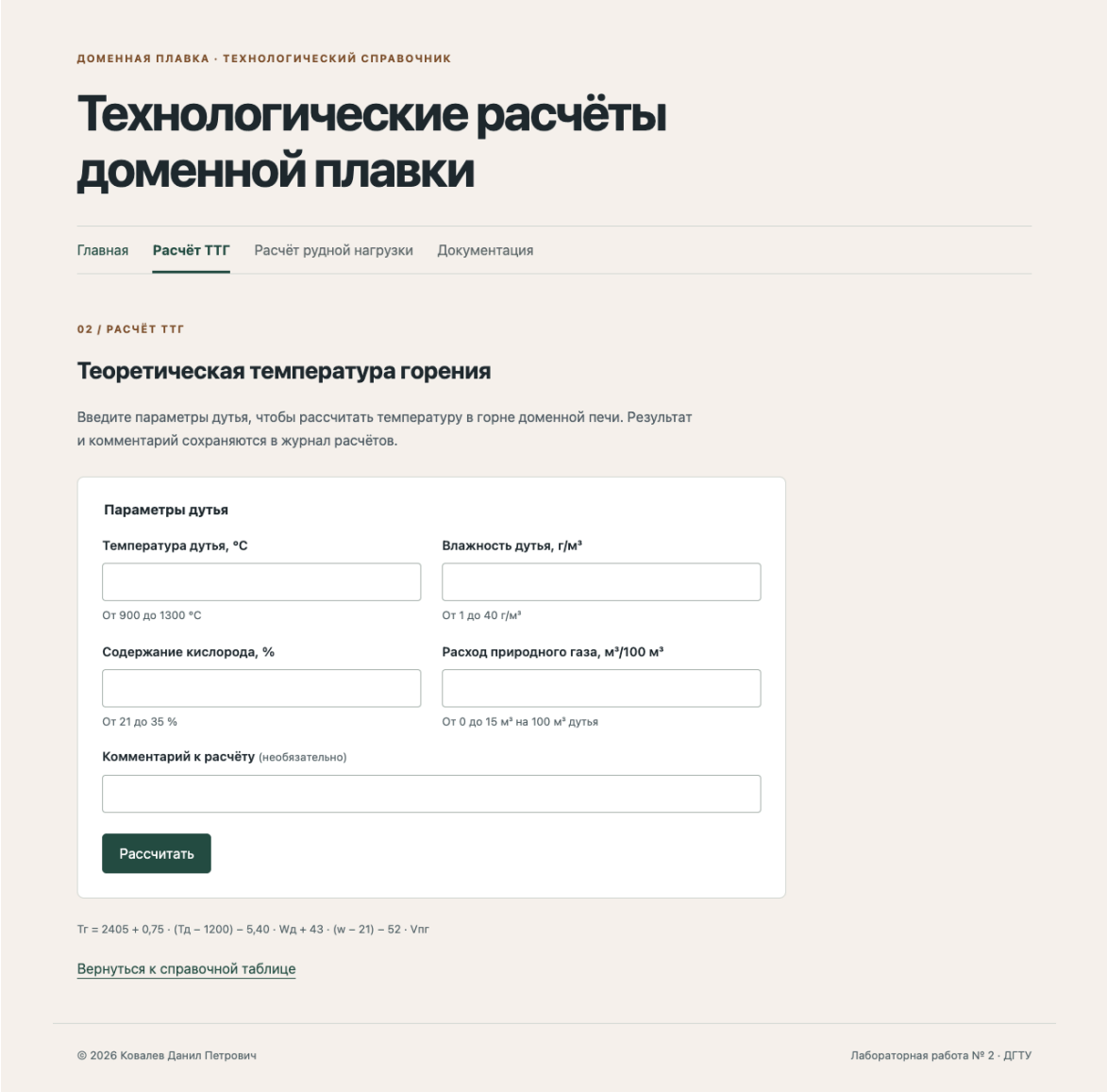


Рисунок 1 — начальное состояние страницы расчёта ТТГ

6.1. Результат контрольного расчёта

После отправки параметров Тд = 1000 °C, Wд = 10 г/м³, w = 21 %, Vпг = 0 получено 2201,0 °C. Введённые параметры и комментарий отображаются на странице, результат дописан в журнал.

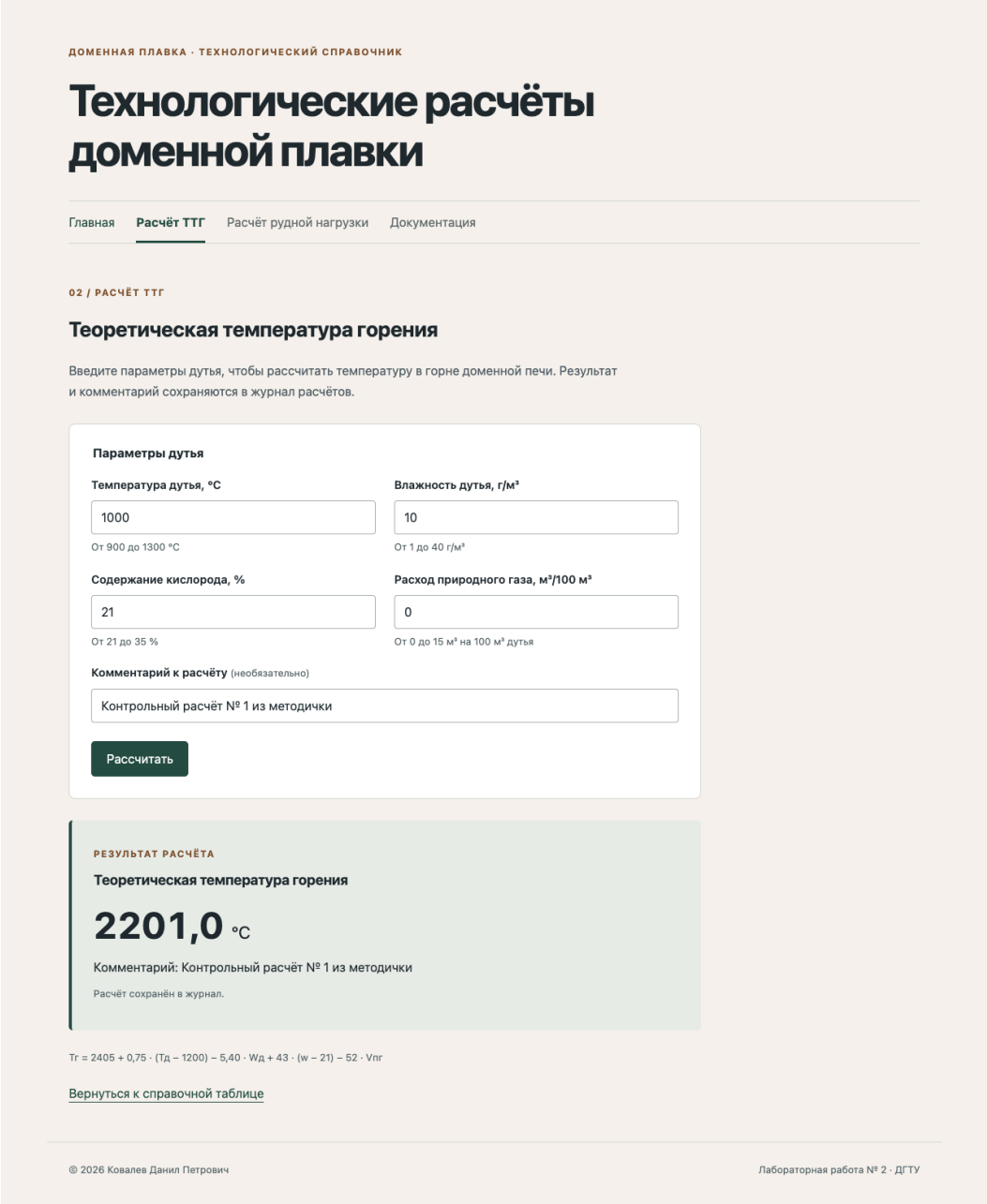


Рисунок 2 — первый контрольный расчёт и подтверждение сохранения

6.2. Обработка неверного ввода

Для проверки серверной валидации вместо температуры введено abc. Сервер вернул HTTP 400, показал сообщение «Температура дутья: введите число», сохранил остальные значения и не выполнил запись в журнал. Отдельно проверены ввод вне диапазонов, NaN, бесконечность и отсутствующие поля.

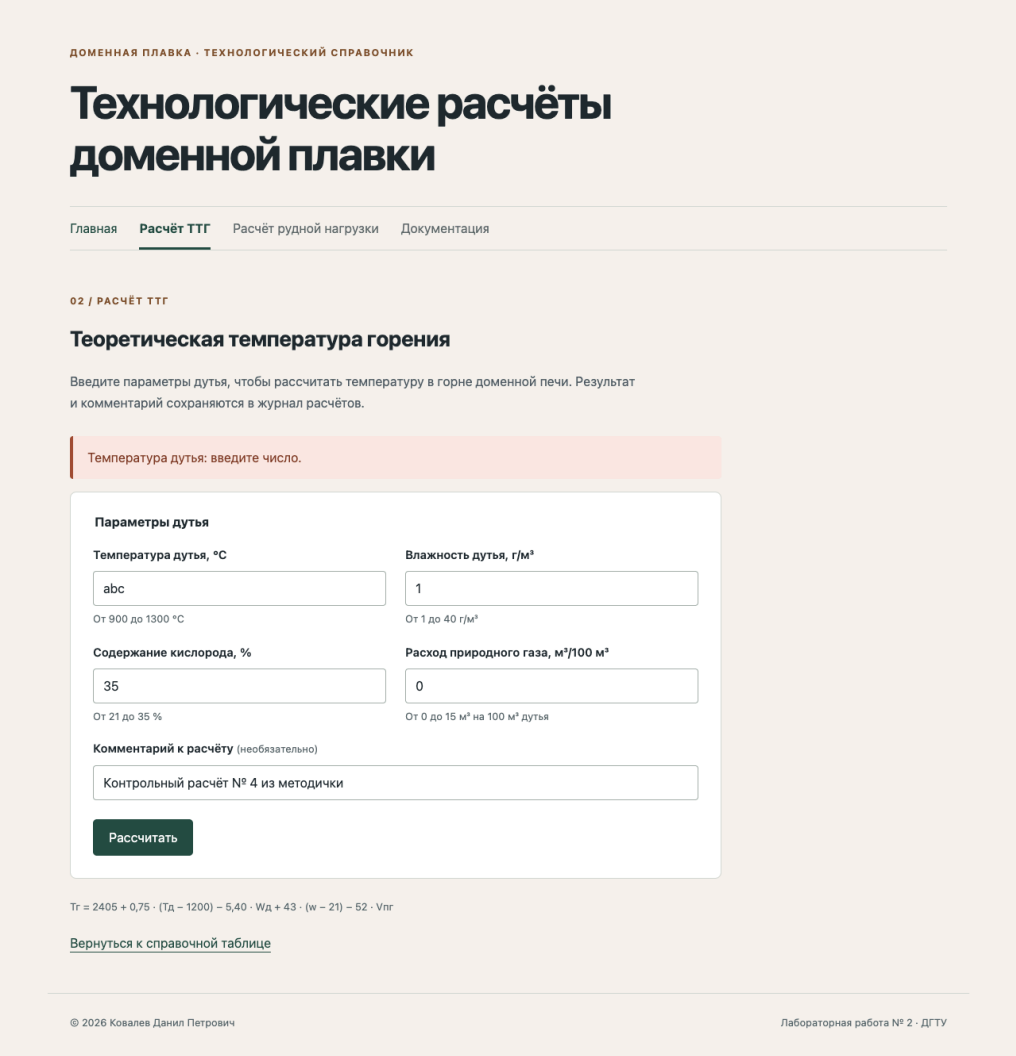


Рисунок 3 — сообщение при вводе нечислового значения

Дополнительно форма и главная страница проверены при ширине 390 px: горизонтальное переполнение отсутствует, расчёт выполняется. Снимки index-mobile.png и tempr-mobile.png находятся в docs/screenshots. Комментарий с тегом script отображён текстом, выполнение JavaScript не произошло.

7. Автоматическая проверка

Выполнены модульные проверки формулы и физического смысла коэффициентов, интеграционные HTTP-тесты формы и журнала, проверки сохранённой главной страницы. Проверки работают с временными файлами и не изменяют пользовательский журнал. Дополнительно выполнены статические проверки и браузерный сценарий.

```
uv run ruff format --check .
uv run ruff check .
uv run mypy
uv run bandit -c pyproject.toml -r src
uv run lint-imports
uv run pytest --cov --cov-report=term-missing
uv run --group report python tools/capture_pages.py
```

Листинг 8 — команды проверки

Проверка	**Фактический результат**
pytest	51 тестов пройдено
Покрытие исходников	93%
4 контрольных значения	Совпали в модульных, HTTP- и браузерных проверках
Ruff / mypy	Проверки пройдены
Bandit	Замечаний не обнаружено
import-linter	4 архитектурных контракта соблюдены
Журнал	Дописывание, создание каталога, экранирование переносов, ошибка записи
Chrome / Playwright	Форма, расчёты, ошибка, экранирование, экран 390 px — успешно
Docker	Сборка и запуск не выполнялись

Протокол команд сохранён в docs/verification.txt, результаты Chrome — в docs/browser-verification.txt. Непокрытые строки относятся к точкам запуска приложения; сама формула, валидация, HTTP-контроллер, сценарий расчёта и файловый адаптер покрыты тестами полностью.

8. Ответы на контрольные вопросы

1. Чем GET отличается от POST и почему выбран POST?

GET запрашивает представление ресурса; параметры обычно передаются в строке URL. POST передаёт данные в теле запроса для обработки. В приложении GET открывает форму без записи в журнал, а POST отправляет параметры, выполняет расчёт и сохраняет запись. POST не обеспечивает шифрование: для защиты передачи нужен HTTPS.

2. Как данные формы попадают в контроллер? Какой атрибут служит ключом?

Браузер сериализует успешные элементы формы и отправляет их на URL из action методом POST. Flask предоставляет поля через request.form. Ключ — атрибут name, например request.form.get('t_blast'). Атрибут id нужен для связи с label, CSS и DOM и не определяет имя передаваемого параметра.

3. Почему данные приходят строками? Что будет без преобразования?

В обычной HTML-форме значения передаются как текст в кодировке формы; числовой тип Python не сохраняется протоколом. Перед вычислением требуется int() либо float(). В данной работе float() поддерживает дроби. Если подставить строку в вычитание t_blast − 1200, Python вызовет TypeError. Некорректный текст вызывает ValueError при преобразовании; сервер обрабатывает эту ошибку и возвращает понятное сообщение.

8.1. Ответы на вопросы 4–5

4. Как значение передаётся в шаблон? Что делает {% if tempr %}?

Контроллер вызывает render_template('tempr.html', tempr=result), а шаблон использует имя tempr. Конструкция {% if tempr %} выводит блок только при истинном значении и скрывает его при первом открытии. Для чисел она также скрывает нулевой результат, поэтому в реализации применено более точное условие {% if tempr is not none %}. Комментарий передаётся вместе со значениями формы и выводится с автоматическим экранированием Jinja.

5. Какие семантические теги использованы и как объясняются знаки коэффициентов?

header содержит заголовок, nav — меню, main — основное содержимое, section — тематический раздел, footer — автора и номер работы. В форме используются form, fieldset, legend, label, input, button и output; в справочнике — table, caption, thead, tbody и th. Положительный коэффициент 0,75 отражает поступление тепла с горячим дутьём. Отрицательный −5,40 соответствует затратам тепла на влагу. Положительный +43 учитывает уменьшение доли балластного азота при обогащении кислородом выше 21 %. Отрицательный −52 связан с эндотермическим разложением метана.

9. Вывод

В лабораторной работе № 2 завершена страница расчёта теоретической температуры горения. В проект первой лабораторной добавлены независимая функция расчёта, модель параметров с проверкой диапазонов, форма ввода и обработчик GET/POST, вывод результата и комментария, а также файловый журнал с дописыванием записей.

Все четыре контрольных значения из методических указаний воспроизведены. Работоспособность подтверждена автоматическими тестами, статическими проверками и сценариями в Chrome, включая мобильную ширину, неверный ввод и экранирование пользовательского комментария. Приложение проверено локально без сборки Docker.

10. Использованные материалы

1. ДГТУ. Лабораторная работа № 2 «Разработка Web-приложения для расчёта теоретической температуры горения». Разработка страницы расчёта температуры. Ростов-на-Дону, 2026. Предоставленный PDF, с. 2–9.

2. Исходный проект и отчёт лабораторной работы № 1: каталог ../1. Ковалев Данил Петрович, ВКБ53.

3. Исходные тексты приложения, тестов и браузерного сценария ЛР2; протоколы docs/verification.txt и docs/browser-verification.txt.